LangChain & LangGraph

Complete End-to-End Theory Notes

Component Structure • Architecture • Installation • Code Examples

LangChain LangGraph Agents LCEL

Beginner to Advanced | Theory + Practical Code | 2024 Edition

Table of Contents

PART 1 — LangChain

The framework for building LLM-powered applications

1.1 Introduction & Philosophy

LangChain is an open-source Python (and JavaScript) framework that provides standardized, composable building blocks for building applications powered by Large Language Models (LLMs). Its core philosophy is **composability** — every component is a *Runnable* that can be chained together using a simple pipe (|) operator.

LangChain solves three key problems: **(1)** Connecting LLMs to external data sources, **(2)** Enabling LLMs to take actions via tools/agents, and **(3)** Managing conversation state and memory across multi-turn interactions.

Key Design Principles

- Modular: swap any component (LLM, vector store, parser) without rewriting logic
- Composable: use the | pipe operator to wire Runnables into pipelines (LCEL)
- Provider-agnostic: works with OpenAI, Anthropic, HuggingFace, Ollama, Cohere, etc.
- Observable: built-in callbacks + LangSmith for tracing every step
- Async-first: every Runnable supports .invoke(), .stream(), .batch(), .astream()

1.2 Installation & Package Structure

LangChain is split into several focused packages. Install only what you need:

Python

```
# Core packages — always needed
pip install langchain # core framework
pip install langchain-core # base Runnables, interfaces
pip install langchain-community # 3rd party integrations

# LLM provider packages (choose your provider)
pip install langchain-openai # OpenAI / Azure OpenAI
pip install langchain-anthropic # Anthropic Claude
pip install langchain-google-genai # Google Gemini
pip install langchain-ollama # Local models via Ollama
pip install langchain-huggingface # HuggingFace models

# Vector stores
pip install chromadb # Chroma (local)
pip install faiss-cpu # FAISS (in-memory)
pip install pinecone-client # Pinecone (cloud)

# Document processing
pip install pypdf # PDF loading
pip install unstructured # HTML, DOCX, etc.
pip install tiktoken # Token counting (OpenAI)

# Agent tools
pip install duckduckgo-search # Web search tool
pip install wikipedia # Wikipedia tool
```

Package Architecture:

Package	Purpose	Import Prefix
`langchain-core`	Base classes: Runnable, BaseMessage, etc.	langchain_core.*
`langchain`	High-level chains, agents, memory	langchain.*
`langchain-community`	100+ integrations (loaders, stores)	langchain_community.*
`langchain-openai`	OpenAI & Azure wrappers	langchain_openai.*
`langchain-anthropic`	Anthropic Claude wrappers	langchain_anthropic.*
`langchain-google-genai`	Google Gemini wrappers	langchain_google_genai.*

1.3 Architecture Overview

LangChain's architecture is layered. At the bottom are **Models** (LLMs), above them are **Prompts** and **Parsers**, then **Chains/LCEL** to compose them, then **Retrievers** for external knowledge, and finally **Agents** for autonomous decision-making.

LangChain Layer Stack (left = foundation, right = highest abstraction)

1.4 Core Components At a Glance

Component	Class / Module	What it Does
LLM	`langchain_openai.OpenAI`	Text-in, text-out completion model
ChatModel	`langchain_openai.ChatOpenAI`	Message list in, AIMessage out
PromptTemplate	`langchain_core.prompts`	Parameterized prompt strings
ChatPromptTemplate	`langchain_core.prompts`	System+Human message prompts
OutputParser	`langchain_core.output_parsers`	Parse LLM text → structured data
Chain (LCEL)	`Runnable | Runnable`	Compose any two Runnables with |
DocumentLoader	`langchain_community.document_loaders`	Load docs from PDF, URL, DB, etc.
TextSplitter	`langchain.text_splitter`	Split docs into overlapping chunks
Embeddings	`langchain_openai.OpenAIEmbeddings`	Convert text → dense float vectors
VectorStore	`langchain_community.vectorstores`	Store+search vectors by similarity
Retriever	`vectorstore.as_retriever()`	Interface to fetch relevant docs
Tool	`@tool decorator / BaseTool`	Function an agent can call
Agent	`create_openai_tools_agent()`	LLM + tools loop (ReAct pattern)
Memory	`langchain.memory.*`	Persist conversation history
Callback	`langchain_core.callbacks`	Hook for logging/tracing/streaming

1.5 Prompt Templates

Prompts are the primary way to communicate with LLMs. LangChain provides template classes that support variable substitution, few-shot examples, and multi-turn message formatting.

Python

```python
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.prompts import FewShotPromptTemplate

# 1. Simple PromptTemplate (string-based)
prompt = PromptTemplate.from_template(
    'Translate the following text to {language}: {text}'
)
formatted = prompt.format(language='French', text='Hello World')

# 2. ChatPromptTemplate (for chat models)
chat_prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert {domain} assistant. Be concise.'),
    ('human', '{question}'),
    ('ai', '{answer}'), # optional example
    ('human', '{followup}'), # actual question
])

# Partial prompts — pre-fill some variables
partial = chat_prompt.partial(domain='Python')

# 3. Few-shot prompting
examples = [
    {'input': 'happy', 'output': 'sad'},
    {'input': 'tall', 'output': 'short'},
]
example_prompt = PromptTemplate.from_template('Input: {input} -> Output: {output}')
few_shot = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix='Give the antonym of each word:',
    suffix='Input: {word} -> Output:',
    input_variables=['word']
)
```

1.6 LLMs & ChatModels

LangChain wraps every LLM provider with a consistent interface. **ChatModels** take a list of *BaseMessage* objects (SystemMessage, HumanMessage, AIMessage) and return an AIMessage. All models support .invoke(), .stream(), .batch(), and async variants.

Python

```python
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_ollama import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage

# OpenAI
llm = ChatOpenAI(
    model='gpt-4o-mini',
    temperature=0, # 0=deterministic, 1=creative
    max_tokens=1000,
    timeout=30,
)

# Anthropic Claude
claude = ChatAnthropic(model='claude-3-5-sonnet-20241022')

# Local via Ollama (no API key needed)
local_llm = ChatOllama(model='llama3.2')

# Basic invocation
messages = [
    SystemMessage(content='You are a helpful assistant.'),
    HumanMessage(content='What is the capital of France?'),
]
response = llm.invoke(messages)
print(response.content) # 'Paris'
print(response.usage_metadata) # token counts

# Streaming
for chunk in llm.stream(messages):
    print(chunk.content, end='', flush=True)

# Bind parameters (model-level config)
llm_json = llm.bind(response_format={'type': 'json_object'})
llm_tools = llm.bind_tools([search_tool, calc_tool])
```

1.7 Output Parsers

Output Parsers transform the raw string output of an LLM into structured Python objects. They also inject formatting instructions into the prompt via *get_format_instructions()*.

Python

```python
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.output_parsers import CommaSeparatedListOutputParser
from langchain_core.pydantic_v1 import BaseModel, Field

# 1. String parser (default — just returns .content)
str_parser = StrOutputParser()

# 2. Comma-separated list
list_parser = CommaSeparatedListOutputParser()
# Adds: 'Your response should be a comma separated list...'

# 3. Pydantic structured output (RECOMMENDED)
class MovieReview(BaseModel):
    title: str = Field(description='Movie title')
    rating: float = Field(description='Rating from 1 to 10')
    pros: list = Field(description='List of pros')
    summary: str = Field(description='One sentence summary')

# Method A: with_structured_output (cleanest)
structured_llm = ChatOpenAI(model='gpt-4o-mini').with_structured_output(MovieReview)
review = structured_llm.invoke('Review the movie Inception')
print(review.rating) # 9.0 (typed!)
print(review.pros) # ['Mind-bending plot', ...]

# Method B: JsonOutputParser in LCEL chain
from langchain_core.prompts import ChatPromptTemplate
parser = JsonOutputParser(pydantic_object=MovieReview)
prompt = ChatPromptTemplate.from_messages([
    ('system', 'Review movies.\n{format_instructions}'),
    ('human', 'Review: {movie}'),
]).partial(format_instructions=parser.get_format_instructions())
chain = prompt | ChatOpenAI() | parser
result = chain.invoke({'movie': 'The Matrix'})
```

1.8 LCEL — LangChain Expression Language

LCEL is LangChain's declarative way to compose Runnables. The **| operator** chains output of one component to input of the next. Every LCEL chain is itself a Runnable, enabling nesting, parallelism, and automatic streaming support.

LCEL Runnable Interface — Methods every component has

- .invoke(input) — run synchronously, return output
- .stream(input) — yield output tokens as they arrive
- .batch([input1, ...]) — run multiple inputs in parallel
- .astream(input) — async version of stream
- .abatch([...]) — async batch
- .with_config(...) — attach callbacks, tags, metadata
- .with_retry(...) — auto-retry on failure
- .with_fallbacks([...]) — fallback chain if primary fails

Python

```python
from langchain_core.runnables import (
    RunnablePassthrough, RunnableParallel, RunnableLambda
)
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o-mini')

# Basic chain — | wires components together
chain = ChatPromptTemplate.from_template('{topic}') | llm | StrOutputParser()
chain.invoke({'topic': 'Explain gravity in one sentence'})

# RunnablePassthrough — pass input unchanged
chain2 = RunnablePassthrough() | llm

# RunnableParallel — run multiple branches simultaneously
parallel = RunnableParallel({
    'summary': prompt | llm | StrOutputParser(),
    'keywords': kw_prompt | llm | list_parser,
    'original': RunnablePassthrough()
})
result = parallel.invoke({'text': 'LangChain is great'})
# result = {'summary': '...', 'keywords': [...], 'original': {...}}

# RunnableLambda — wrap any Python function
upper = RunnableLambda(lambda x: x.upper())
chain3 = chain | upper

# Fallbacks — use backup if primary fails
robust_llm = ChatOpenAI(model='gpt-4o').with_fallbacks([
    ChatOpenAI(model='gpt-4o-mini')
])

# Retry — auto-retry on rate limits
retry_llm = llm.with_retry(stop_after_attempt=3)
```

1.9 Document Loaders & Text Splitters

Document Loaders read data from many sources and return a list of *Document* objects (each has **.page_content** string and **.metadata** dict). Text Splitters then chunk documents into smaller pieces for embedding.

Python

```python
# ■■ LOADERS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_community.document_loaders import (
    PyPDFLoader, # PDF files
    WebBaseLoader, # Web pages (HTML)
    TextLoader, # Plain .txt files
    CSVLoader, # CSV files
    JSONLoader, # JSON with jq schema
    DirectoryLoader, # Load all files in a folder
    YoutubeLoader, # YouTube transcripts
    NotionDirectoryLoader, # Notion exports
)

loader = PyPDFLoader('handbook.pdf')
docs = loader.load() # returns List[Document]
print(docs[0].page_content[:200])
print(docs[0].metadata) # {'source': 'handbook.pdf', 'page': 0}

# Load all PDFs from a directory
dir_loader = DirectoryLoader('./docs/', glob='**/*.pdf',
                      loader_cls=PyPDFLoader)
all_docs = dir_loader.load()

# ■■ TEXT SPLITTERS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter, # RECOMMENDED — tries \n\n first
    CharacterTextSplitter, # splits on single separator
    TokenTextSplitter, # splits by token count
    MarkdownHeaderTextSplitter, # splits by # headers
)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, # max chars per chunk
    chunk_overlap=200, # overlap to preserve context across chunks
    length_function=len, # can use tiktoken for token-based sizing
    separators=['\n\n', '\n', ' ', ''] # tries each in order
)
chunks = splitter.split_documents(docs)
print(f'{len(chunks)} chunks created')
print(chunks[0].page_content) # first chunk text
```

1.10 Embeddings & Vector Stores

Embeddings convert text into high-dimensional float vectors that capture semantic meaning. Similar texts have similar (close) vectors. Vector stores persist these vectors and support **similarity search** to find the most semantically relevant documents for a query.

Python

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma, FAISS

# ■■ EMBEDDINGS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# OpenAI (paid, high quality)
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
# Dimensions: small=1536, large=3072

# HuggingFace (free, runs locally)
hf_embeddings = HuggingFaceEmbeddings(
    model_name='sentence-transformers/all-MiniLM-L6-v2'
    # 384-dim, fast, great for most use cases
)

# Embed a single string
vector = embeddings.embed_query('What is machine learning?')
print(len(vector)) # 1536

# ■■ CHROMA (local persistent vector store) ■■■■■■■■■■■■■■■■■■■■■■■■■
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='./chroma_db',
    collection_name='my_docs'
)

# Load existing Chroma store
vectorstore = Chroma(
    persist_directory='./chroma_db',
    embedding_function=embeddings
)

# ■■ FAISS (in-memory, fast) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
faiss_store = FAISS.from_documents(chunks, embeddings)
faiss_store.save_local('./faiss_index') # save to disk
faiss_store = FAISS.load_local('./faiss_index', embeddings) # reload

# ■■ SEARCH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
results = vectorstore.similarity_search('How does RAG work?', k=4)
results_with_scores = vectorstore.similarity_search_with_score('query', k=4)
# score = L2 distance (lower = more similar for FAISS)
# score = cosine distance for Chroma
```

1.11 Retrievers

A Retriever is an abstraction layer over a vector store. It implements a standard interface (.invoke(query) → List[Document]) used by chains and agents. Many retrieval strategies exist beyond simple similarity search.

Python

```python
# ■■ BASIC RETRIEVER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
retriever = vectorstore.as_retriever(
    search_type='similarity', # or 'mmr', 'similarity_score_threshold'
    search_kwargs={
        'k': 4, # return top 4 documents
        'filter': {'source': 'manual.pdf'}, # metadata filter
    }
)
docs = retriever.invoke('What is LangChain?')

# ■■ MMR (Maximal Marginal Relevance) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Balances relevance AND diversity — avoids redundant chunks
mmr_retriever = vectorstore.as_retriever(
    search_type='mmr',
    search_kwargs={'k': 4, 'fetch_k': 20, 'lambda_mult': 0.5}
)

# ■■ MULTI-QUERY RETRIEVER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# LLM generates multiple phrasings of the question
from langchain.retrievers import MultiQueryRetriever
mq_retriever = MultiQueryRetriever.from_llm(
    retriever=retriever, llm=ChatOpenAI()
)

# ■■ CONTEXTUAL COMPRESSION RETRIEVER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Compresses/filters retrieved docs to only relevant parts
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor
compressor = LLMChainExtractor.from_llm(ChatOpenAI())
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor, base_retriever=retriever
)

# ■■ ENSEMBLE RETRIEVER (BM25 + Vector) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
bm25 = BM25Retriever.from_documents(chunks) # keyword search
ensemble = EnsembleRetriever(
    retrievers=[bm25, retriever], weights=[0.4, 0.6]
)
```

1.12 Chains (Legacy + LCEL)

LangChain has two styles of chains: the older class-based chains (still widely used) and modern LCEL pipe-based chains. Both accomplish the same goals.

Python

```python
# ■■ LCEL RAG CHAIN (modern, recommended) ■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

rag_prompt = ChatPromptTemplate.from_template('''
Answer the question using ONLY the context below.
If you don't know, say 'I don't know'.
Context: {context}
Question: {question}
Answer:''')

def format_docs(docs):
   return '\n\n'.join(d.page_content for d in docs)

rag_chain = (
   RunnableParallel({
      'context': retriever | format_docs,
      'question': RunnablePassthrough()
   })
   | rag_prompt
   | ChatOpenAI(model='gpt-4o-mini')
   | StrOutputParser()
)
answer = rag_chain.invoke('What is LCEL?')

# ■■ LEGACY RetrievalQA CHAIN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain.chains import RetrievalQA
qa = RetrievalQA.from_chain_type(
   llm=ChatOpenAI(), retriever=retriever,
   chain_type='stuff', # stuff / map_reduce / refine / map_rerank
   return_source_documents=True
)
result = qa.invoke({'query': 'Explain embeddings'})
print(result['result']) # answer
print(result['source_documents']) # retrieved chunks
```

1.13 Agents & Tools

Agents use an LLM as a reasoning engine to decide *which tool to call*, call it, observe the result, and repeat — until the task is complete. This is the ReAct (Reasoning + Acting) pattern. LangGraph is now the preferred way to build custom agents.

Python

```python
from langchain.agents import create_openai_tools_agent, AgentExecutor
from langchain.tools import tool
from langchain_community.tools import DuckDuckGoSearchRun, WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# ■■ DEFINE TOOLS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@tool
def get_stock_price(ticker: str) -> str:
    '''Get current stock price for a ticker symbol like AAPL or GOOGL.'''
    return f'{ticker}: $150.25 (mock data)'

@tool
def calculate(expression: str) -> str:
    '''Evaluate a mathematical expression like 2+2 or 10*5.'''
    try:
        return str(eval(expression))
    except Exception as e:
        return f'Error: {e}'

# Built-in tools
search = DuckDuckGoSearchRun()
wiki = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())

tools = [search, wiki, get_stock_price, calculate]

# ■■ AGENT PROMPT (must have agent_scratchpad placeholder) ■■■■■■■■■■
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful AI with access to tools. Use them to answer.'),
    MessagesPlaceholder('chat_history', optional=True),
    ('human', '{input}'),
    MessagesPlaceholder('agent_scratchpad'), # REQUIRED for tool calls
])

# ■■ CREATE AGENT + EXECUTOR ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
agent = create_openai_tools_agent(llm, tools, prompt)
executor = AgentExecutor(
    agent=agent, tools=tools,
    verbose=True, # prints each step
    max_iterations=10, # safety limit
    handle_parsing_errors=True,
)

result = executor.invoke({'input': 'What is 25 * 48 and search for LangChain news'})
print(result['output'])
```

1.14 Memory

Memory allows an LLM to remember previous conversation turns. LangChain provides several strategies — from simple buffer (store all messages) to summary (compress old messages).

Python

```python
# ■■ BUFFER MEMORY (keeps all messages) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain.memory import ConversationBufferMemory
from langchain.memory import ConversationBufferWindowMemory
from langchain.memory import ConversationSummaryMemory

memory = ConversationBufferWindowMemory(
    k=5, # keep only last 5 turns
    return_messages=True, # return as Message objects (for chat)
    memory_key='chat_history'
)

# ■■ MODERN LCEL MEMORY with RunnableWithMessageHistory ■■■■■■■■■■■■
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_community.chat_message_histories import RedisChatMessageHistory

# In-memory store (dev)
store = {}
def get_session(session_id: str) -> ChatMessageHistory:
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

# Redis store (production)
def get_redis_session(session_id: str):
    return RedisChatMessageHistory(session_id, url='redis://localhost:6379')

chat_prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder('history'),
    ('human', '{input}'),
])
chain = chat_prompt | ChatOpenAI() | StrOutputParser()

chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session,
    input_messages_key='input',
    history_messages_key='history',
)

# Each call with same session_id shares history
cfg = {'configurable': {'session_id': 'user_42'}}
chain_with_history.invoke({'input': 'My name is Alice'}, config=cfg)
chain_with_history.invoke({'input': 'What is my name?'}, config=cfg)
# → 'Your name is Alice'
```

1.15 Callbacks & Tracing (LangSmith)

Callbacks are hooks that fire at each lifecycle event (chain start/end, LLM call, tool use). LangSmith is Anthropic's observability platform that automatically traces every step.

Python

```python
import os
# ■■ LANGSMITH SETUP (set env vars before running) ■■■■■■■■■■■■■■■■■■
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGCHAIN_API_KEY'] = 'ls__your_key_here'
os.environ['LANGCHAIN_PROJECT'] = 'my-project'
# Now every chain.invoke() is automatically traced!

# ■■ CUSTOM CALLBACK HANDLER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_core.callbacks import BaseCallbackHandler

class MyLogger(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        print(f'LLM called with: {prompts[0][:80]}...')

    def on_llm_end(self, response, **kwargs):
        print(f'LLM responded: {response.generations[0][0].text[:80]}')

    def on_tool_start(self, serialized, input_str, **kwargs):
        print(f'Tool {serialized["name"]} called with: {input_str}')

    def on_chain_error(self, error, **kwargs):
        print(f'ERROR: {error}')

# Attach to any chain
chain.invoke({'question': 'Hello'}, config={'callbacks': [MyLogger()]})

# ■■ STREAMING CALLBACK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_core.callbacks import StreamingStdOutCallbackHandler
streaming_llm = ChatOpenAI(streaming=True,
                    callbacks=[StreamingStdOutCallbackHandler()])
```

PART 2 — LangGraph

Build stateful, multi-step agent workflows as directed graphs

2.1 Introduction & Philosophy

LangGraph is a library built on top of LangChain for building **stateful, cyclic, multi-actor workflows**. While LangChain chains are linear (A→B→C), LangGraph models workflows as **directed graphs** with nodes (computation) and edges (flow control) — supporting loops, branching, parallel execution, and human intervention.

LangGraph is especially suited for: **ReAct agents** that loop until done, **multi-agent systems** where agents coordinate, **workflows with approval steps**, and **long-running tasks** that need to be paused, inspected, and resumed.

When to use LangGraph instead of plain LangChain Agents

- You need CYCLES — agent loops back until a condition is met
- You need BRANCHING — different paths based on intermediate results
- You need HUMAN-IN-THE-LOOP — pause for approval before continuing
- You need PERSISTENCE — save and resume long-running workflows
- You need PARALLEL NODES — run multiple agents simultaneously
- You need VISIBILITY — inspect/modify state at every step
- You need MULTI-AGENT — coordinate specialized sub-agents

2.2 Installation

Python

```
# Core LangGraph
pip install langgraph

# Checkpointers for persistence
pip install langgraph-checkpoint # base
pip install langgraph-checkpoint-sqlite # SQLite (local prod)
pip install langgraph-checkpoint-postgres # PostgreSQL (cloud prod)

# LangGraph Platform (for deployment)
pip install langgraph-sdk # client SDK

# Full stack (with LangChain + OpenAI)
pip install langgraph langchain-openai langchain-core

# Verify installation
python -c 'import langgraph; print(langgraph.__version__)'
```

2.3 Core Concepts

Concept	Definition
StateGraph	The main graph object. You add nodes and edges, then compile it.
State	A TypedDict shared by all nodes. The 'memory' of the graph.
Node	A Python function that receives State and returns a partial State update.

Edge	A connection from one node to another (or START/END).
Conditional Edge	An edge where a router function decides which node to go to next.
START	Special constant marking the graph entry point.
END	Special constant marking a terminal state (graph stops).
Checkpointer	Persists graph state after each node — enables pause/resume.
Config	Runtime config dict with thread_id, callbacks, recursion_limit.
Interrupt	Pause execution before/after a node for human review/approval.
Subgraph	A compiled graph used as a node inside another graph.

2.4 StateGraph & State Definition

The **State** is a TypedDict that acts as the shared memory flowing through the graph. Each node receives the full state and returns a dictionary with only the keys it updates. Using **Annotated** with reducers controls how updates are merged.

Python

```python
from typing import TypedDict, Annotated, Sequence, List, Optional
from langchain_core.messages import BaseMessage
from langgraph.graph import StateGraph, START, END
import operator

# ■■ STATE DEFINITION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class AgentState(TypedDict):
    # Annotated[list, operator.add] = APPEND new items (don't overwrite)
    messages: Annotated[Sequence[BaseMessage], operator.add]
    # Plain fields are OVERWRITTEN by the last node to set them
    question: str
    context: List[str]
    answer: str
    iterations: int
    error: Optional[str]

# ■■ GRAPH CREATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
graph = StateGraph(AgentState)

# ■■ ADD NODES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Node = function(state) -> dict of updates
def my_node(state: AgentState) -> dict:
    return {'answer': 'computed answer', 'iterations': state['iterations'] + 1}

graph.add_node('node_name', my_node)

# ■■ ADD EDGES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
graph.add_edge(START, 'node_name') # entry point
graph.add_edge('node_name', END) # exit
graph.add_edge('node_a', 'node_b') # a always goes to b

# ■■ COMPILE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
app = graph.compile()

# ■■ INVOKE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
initial_state = {'question': 'What is AI?', 'iterations': 0}
final_state = app.invoke(initial_state)
print(final_state['answer'])
```

2.5 Nodes — Types & Patterns

Nodes are the computation units of LangGraph. They can be plain Python functions, LangChain Runnables, or prebuilt nodes like ToolNode.

Python

```python
from langgraph.prebuilt import ToolNode, tools_condition
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

# ■■ PATTERN 1: Plain function node ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def retriever_node(state: AgentState) -> dict:
    docs = retriever.invoke(state['question'])
    context = [d.page_content for d in docs]
    return {'context': context}

# ■■ PATTERN 2: LLM node ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(model='gpt-4o-mini').bind_tools(tools)

def llm_node(state: AgentState) -> dict:
    response = llm.invoke(state['messages'])
    return {'messages': [response]} # appended via operator.add

# ■■ PATTERN 3: Prebuilt ToolNode ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Automatically handles all tool_calls in the last AIMessage
tool_node = ToolNode(tools) # returns ToolMessages

# ■■ PATTERN 4: Node with error handling ■■■■■■■■■■■■■■■■■■■■■■■■■■
def safe_node(state: AgentState) -> dict:
    try:
        result = risky_operation(state['question'])
        return {'answer': result, 'error': None}
    except Exception as e:
        return {'error': str(e), 'answer': ''}

# ■■ PATTERN 5: Async node ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async def async_node(state: AgentState) -> dict:
    response = await llm.ainvoke(state['messages'])
    return {'messages': [response]}

# Adding nodes to graph
graph.add_node('retrieve', retriever_node)
graph.add_node('llm', llm_node)
graph.add_node('tools', tool_node)
graph.add_node('safe_work', safe_node)
```

2.6 Edges & Conditional Routing

Edges define the flow between nodes. **Normal edges** always go to the same next node. **Conditional edges** use a router function to dynamically pick the next node based on the current state — enabling branching, loops, and error recovery.

Python

```python
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import tools_condition

# ■■ NORMAL EDGES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
graph.add_edge(START, 'first_node') # entry
graph.add_edge('node_a', 'node_b') # unconditional flow
graph.add_edge('node_b', END) # exit

# ■■ CONDITIONAL EDGES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Router function: receives state, returns node name (str) or END
def route_after_llm(state: AgentState) -> str:
    last_msg = state['messages'][-1]
    if last_msg.tool_calls: # LLM wants to use a tool
        return 'tools'
    if state.get('error'): # there was an error
        return 'error_handler'
    return END # we're done

graph.add_conditional_edges(
    'llm', # from this node
    route_after_llm, # router function
    # Optional: mapping dict for explicit routing
    {'tools': 'tools', 'error_handler': 'error_handler', END: END}
)

# ■■ PREBUILT tools_condition ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Checks if last message has tool_calls → 'tools', else → END
graph.add_conditional_edges('llm', tools_condition)

# ■■ LOOPING BACK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def should_continue(state: AgentState) -> str:
    if state['iterations'] < 3 and not state.get('answer'):
        return 'llm' # loop back
    return END

graph.add_conditional_edges('evaluator', should_continue)
graph.add_edge('tools', 'llm') # after tools, always back to llm

# ■■ PARALLEL (fan-out) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Add same source to multiple destinations
graph.add_edge('start_node', 'branch_a')
graph.add_edge('start_node', 'branch_b') # both run in parallel
# Both branches feed into a merge node
graph.add_edge('branch_a', 'merge_node')
graph.add_edge('branch_b', 'merge_node')
```

2.7 Checkpointers & Persistence

Checkpointers save the complete graph state after every node execution. This enables **pause and resume**, **time-travel debugging** (roll back to any past state), and **multi-session continuity** (different users, same thread_id shares state).

Python

```python
from langgraph.checkpoint.memory import MemorySaver
# pip install langgraph-checkpoint-sqlite
from langgraph.checkpoint.sqlite import SqliteSaver
# pip install langgraph-checkpoint-postgres
from langgraph.checkpoint.postgres import PostgresSaver
import sqlite3

# ■■ IN-MEMORY (dev only, lost on restart) ■■■■■■■■■■■■■■■■■■■■■■■■■
memory = MemorySaver()
app = graph.compile(checkpointer=memory)

# ■■ SQLITE (local production) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
conn = sqlite3.connect('checkpoints.db', check_same_thread=False)
sqlite_cp = SqliteSaver(conn)
app = graph.compile(checkpointer=sqlite_cp)

# ■■ POSTGRES (cloud production) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
with PostgresSaver.from_conn_string('postgresql://user:pw@host/db') as pg_cp:
    app = graph.compile(checkpointer=pg_cp)

# ■■ USING THREADS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# thread_id isolates state per conversation/user
config = {'configurable': {'thread_id': 'user_123_session_1'}}

# First invocation — starts fresh
result1 = app.invoke({'messages': [HumanMessage('Hi')]}, config=config)

# Second invocation — continues from where we left off
result2 = app.invoke({'messages': [HumanMessage('What did I say?')]}, config=config)

# ■■ INSPECT STATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
current = app.get_state(config)
print(current.values) # current state dict
print(current.next) # which node runs next
print(current.config) # checkpoint metadata

# Get full history (all past states)
for state in app.get_state_history(config):
    print(state.config['configurable']['checkpoint_id'])

# Time-travel: restart from a past checkpoint
past_config = {'configurable': {'thread_id': '...', 'checkpoint_id': '...past...'}}
app.invoke({'messages': []}, config=past_config)
```

2.8 Human-in-the-Loop

LangGraph supports pausing execution before or after any node to allow human review, approval, editing, or correction. This requires a checkpointer.

Python

```python
# ■■ INTERRUPT BEFORE a node ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
app = graph.compile(
    checkpointer=MemorySaver(),
    interrupt_before=['tools'], # pause before calling tools
    # interrupt_after=['llm'],     # or pause after LLM responds
)

config = {'configurable': {'thread_id': 'review_session_1'}}

# Run until the interrupt
result = app.invoke(
    {'messages': [HumanMessage('Search for AI news and summarize')]},
    config=config
)
# Execution is paused — result contains state up to the interrupt

# ■■ INSPECT WHAT WOULD HAPPEN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
state = app.get_state(config)
print('Next node:', state.next) # ('tools',)
print('Pending tools:', state.values['messages'][-1].tool_calls)

# ■■ OPTION A: Approve — resume with None ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
final = app.invoke(None, config=config)

# ■■ OPTION B: Edit state before resuming ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Override the tool call the agent planned
from langchain_core.messages import AIMessage
app.update_state(
    config,
    {'messages': [AIMessage(content='Modified by human', tool_calls=[])]}
)
final = app.invoke(None, config=config)

# ■■ OPTION C: Restart from a different node ■■■■■■■■■■■■■■■■■■■■■■■■■■
app.update_state(config, {'question': 'new question'}, as_node='llm')

# ■■ DYNAMIC BREAKPOINTS with NodeInterrupt ■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langgraph.errors import NodeInterrupt

def human_approval_node(state):
    if state.get('needs_approval'):
        raise NodeInterrupt(f"Approve action: {state['planned_action']}")
    return {}
```

2.9 Subgraphs & Modularity

A compiled LangGraph app can be used as a **node inside another graph** — called a subgraph. This enables modular design: build specialized agents, then compose them into a larger orchestrator.

Python

```python
# ■■ DEFINE A SUBGRAPH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class SubState(TypedDict):
    sub_input: str
    sub_output: str

sub_graph = StateGraph(SubState)
sub_graph.add_node('process', lambda s: {'sub_output': s['sub_input'].upper()})
sub_graph.add_edge(START, 'process')
sub_graph.add_edge('process', END)
sub_app = sub_graph.compile() # compiled subgraph

# ■■ USE AS A NODE IN PARENT GRAPH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class ParentState(TypedDict):
    input: str
    result: str

def call_subgraph(state: ParentState) -> dict:
    sub_result = sub_app.invoke({'sub_input': state['input']})
    return {'result': sub_result['sub_output']}

parent = StateGraph(ParentState)
parent.add_node('subgraph_node', call_subgraph)
parent.add_edge(START, 'subgraph_node')
parent.add_edge('subgraph_node', END)
parent_app = parent.compile()

# ■■ DIRECT SUBGRAPH (shared state keys) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# If parent and sub share state keys, add subgraph directly as node
parent.add_node('my_subgraph', sub_app) # sub_app as a node
```

2.10 Multi-Agent Architectures

LangGraph is the ideal framework for multi-agent systems. Common patterns include: **Supervisor** (one agent orchestrates others), **Swarm** (agents hand off to peers), and **Hierarchical** (nested supervisor trees).

Python

```python
# ■■ SUPERVISOR PATTERN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# One supervisor LLM decides which specialist agent to call next

from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

agents = ['researcher', 'analyst', 'writer']
supervisor_prompt = ChatPromptTemplate.from_messages([
   ('system', f'''You are a supervisor managing these agents: {agents}.
    Given the task, pick the next agent to act, or FINISH if done.
    Respond with JSON: {{"next": "agent_name or FINISH"}}'''),
   ('human', 'Task: {task}\nHistory: {history}'),
])

class SupervisorState(TypedDict):
   task: str
   history: Annotated[list, operator.add]
   next_agent: str
   final_answer: str

def supervisor(state: SupervisorState) -> dict:
   llm = ChatOpenAI(model='gpt-4o-mini')
   response = llm.invoke(supervisor_prompt.format_messages(
      task=state['task'], history=state['history']
   ))
   import json
   decision = json.loads(response.content)
   return {'next_agent': decision['next']}

def researcher_agent(state): ...
def analyst_agent(state): ...
def writer_agent(state): ...

# Router
def route_to_agent(state) -> str:
   n = state['next_agent']
   return END if n == 'FINISH' else n

# Build graph
multi = StateGraph(SupervisorState)
multi.add_node('supervisor', supervisor)
multi.add_node('researcher', researcher_agent)
multi.add_node('analyst', analyst_agent)
multi.add_node('writer', writer_agent)

multi.add_edge(START, 'supervisor')
multi.add_conditional_edges('supervisor', route_to_agent)
for agent in ['researcher', 'analyst', 'writer']:
   multi.add_edge(agent, 'supervisor') # always report back

app = multi.compile(checkpointer=MemorySaver())
```

2.11 Streaming

Python

```python
# ■■ STREAM GRAPH EVENTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
config = {'configurable': {'thread_id': 'stream_test'}}

# stream() yields each node's output as it completes
for event in app.stream(
    {'messages': [HumanMessage('Tell me about AI')]},
    config=config,
    stream_mode='values' # yields full state after each node
):
    print(event) # full state dict

# stream_mode='updates' — only the delta (what changed)
for chunk in app.stream(input, config, stream_mode='updates'):
    node_name = list(chunk.keys())[0]
    print(f'{node_name}: {chunk[node_name]}')

# ■■ STREAM LLM TOKENS inside a node ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Use astream_events for fine-grained token streaming
async def stream_tokens():
    async for event in app.astream_events(
        {'messages': [HumanMessage('Hello')]},
        config=config,
        version='v2'
    ):
        kind = event['event']
        if kind == 'on_chat_model_stream':
            chunk = event['data']['chunk']
            print(chunk.content, end='', flush=True)
        elif kind == 'on_tool_start':
            print(f'\nCalling tool: {event["name"]}')
        elif kind == 'on_tool_end':
            print(f'Tool result: {event["data"]["output"]}')
```

2.12 Production Patterns & Best Practices

Python

```python
# ■■ RECURSION LIMIT ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Prevent infinite loops
app = graph.compile()
config = {'recursion_limit': 25} # default is 25
result = app.invoke(input, config=config)

# ■■ ERROR RECOVERY NODE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def error_handler(state: AgentState) -> dict:
    print(f'Recovering from: {state["error"]}')
    return {'error': None, 'iterations': 0}

def route_error(state) -> str:
    if state.get('error'):
        return 'error_handler'
    return 'continue_node'

# ■■ ASYNC GRAPH (for FastAPI integration) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from fastapi import FastAPI
from langchain_core.messages import HumanMessage
from langgraph.graph import StateGraph

app_api = FastAPI()

@app_api.post('/chat')
async def chat(message: str, session_id: str):
    config = {'configurable': {'thread_id': session_id}}
    result = await lg_app.ainvoke(
        {'messages': [HumanMessage(message)]}, config=config
    )
    return {'response': result['messages'][-1].content}

@app_api.get('/stream')
async def stream_chat(message: str):
    from fastapi.responses import StreamingResponse
    async def generate():
        async for event in lg_app.astream_events(
            {'messages': [HumanMessage(message)]}, version='v2'
        ):
            if event['event'] == 'on_chat_model_stream':
                yield event['data']['chunk'].content
    return StreamingResponse(generate(), media_type='text/plain')
```

PART 3 — Comparison & Cheat Sheet

LangChain vs LangGraph — When to Use Which

Feature	LangChain (LCEL)	LangGraph
Flow type	Linear pipeline	Cyclic graph (DAG + cycles)
State management	Pass-through (no built-in)	Typed shared State (TypedDict)
Loops	Not supported	Native — add_edge back
Branching	Limited (router chain)	Conditional edges, router funcs
Persistence	Manual	Built-in Checkpointers
Human-in-the-loop	Not built-in	interrupt_before/after
Streaming	.stream() tokens	.stream() node events + tokens
Multi-agent	Difficult	First-class (subgraphs)
Debugging	LangSmith traces	State inspection + time-travel
Learning curve	Low	Medium
Best for	Simple chains, RAG, Q&A	Agents, workflows, multi-step

Quick Cheat Sheet — Most Used Imports

Python
```python
# ■■ LANGCHAIN CORE IMPORTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_core.prompts import ChatPromptTemplate, PromptTemplate
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from langchain_core.documents import Document

# ■■ LANGCHAIN MODELS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_anthropic import ChatAnthropic
from langchain_ollama import ChatOllama

# ■■ LANGCHAIN TOOLS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma, FAISS
from langchain.tools import tool
from langchain.agents import create_openai_tools_agent, AgentExecutor
from langchain.memory import ConversationBufferWindowMemory
from langchain_core.runnables.history import RunnableWithMessageHistory

# ■■ LANGGRAPH IMPORTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.errors import NodeInterrupt
from typing import TypedDict, Annotated, Sequence
import operator
```

Environment Variables Reference

```python
import os

# ■■ LLM API Keys ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
os.environ['OPENAI_API_KEY'] = 'sk-...'
os.environ['ANTHROPIC_API_KEY'] = 'sk-ant-...'
os.environ['GOOGLE_API_KEY'] = 'AIza...'

# ■■ LangSmith Tracing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGCHAIN_API_KEY'] = 'ls__...'
os.environ['LANGCHAIN_PROJECT'] = 'my-project-name'
os.environ['LANGCHAIN_ENDPOINT'] = 'https://api.smith.langchain.com'

# ■■ Vector Store Keys ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
os.environ['PINECONE_API_KEY'] = 'pcsk_...'
os.environ['PINECONE_ENVIRONMENT'] = 'gcp-starter'

# ■■ Or use .env file + python-dotenv ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# pip install python-dotenv
from dotenv import load_dotenv
load_dotenv() # reads .env file in current directory
```

Official Documentation

LangChain: python.langchain.com/docs

LangGraph: langchain-ai.github.io/langgraph

LCEL Guide: python.langchain.com/docs/expression_language

LangSmith: docs.smith.langchain.com

LangChain Hub (prompts): hub.langchain.com

Recommended Learning Path

1. Install packages → 2. Build a simple LCEL chain → 3. Add a retriever (RAG) → 4. Add tools (Agent) → 5. Move to LangGraph for stateful workflows → 6. Add persistence + HITL → 7. Build multi-agent system